DAY 4 | WEBSOCKETS | SOCKET.IO

Node.js: Everything You Need to Know

Day 4: Real-Time Applications with Socket.IO

Learn how to extend the modular Express API you built on Day 3 into a fully collaborative, real-time task board. We'll cover the WebSocket protocol, Socket.IO's event model, room-based broadcasting, and the architectural decisions that separate one-off demos from production-ready real-time systems.

By **Wesam Abousaid**

Table of Contents

LEARNING OBJECTIVES

By the End of Day 4, You Will Be Able To...

- Explain the difference between HTTP polling, long-polling, and WebSockets
- Set up Socket.IO on an Express server with no disruption to existing REST routes
- Organise connections using rooms
- Handle connection errors and disconnection events gracefully
- Emit and listen to named events on both server and client
- Broadcast task changes to all connected clients in real time
- Understand the layered architecture: REST for persistence, sockets for live updates

OVERVIEW · 00

The Big Picture: REST + WebSockets

Day 3 ended with a fully modular Express REST API. Today we keep it intact. The key insight: REST and WebSockets solve different problems. They should coexist, not replace each other.

GET /tasks Fetch all tasks on page load (REST — Day 3)	POST /tasks Persist a new task reliably (REST — Day 3)
socket.emit('task:created') Notify everyone a task was added (Socket.IO — today)	socket.emit('task:deleted') Notify everyone a task was deleted (Socket.IO — today)

> "HTTP is a conversation. WebSockets are a live broadcast."

- HTTP: client asks, server answers, connection closes.
- WebSocket: connection stays open — either side can speak at any time.

BLOCK 1 · 15 MIN

Real-Time Web Architecture

Why Polling Breaks Down

Before WebSockets existed, developers faked real-time with polling: the client asks the server every few seconds, 'anything new?' This works at small scale, but the cost compounds quickly.

1. Short Polling

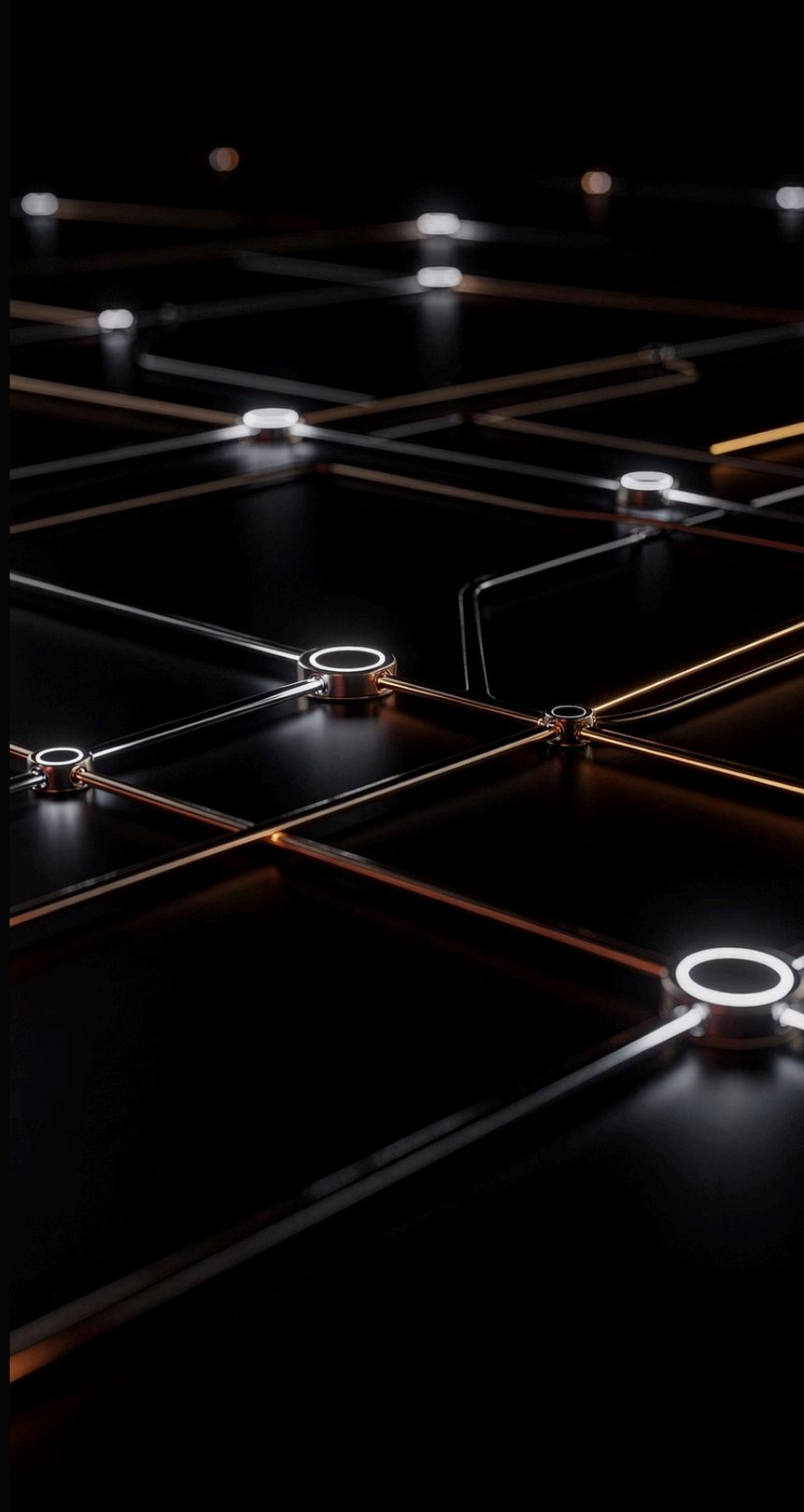
Simple to implement. Wastes bandwidth even when nothing changed. Latency = polling interval.

2. Long Polling

Lower latency. Server holds many open connections (resource-heavy). Complex reconnection logic.

3. WebSockets

Single persistent connection. Sub-millisecond latency. Either side initiates — no request/response cycle.



BLOCK 1 · REAL-TIME ARCHITECTURE

The WebSocket Upgrade Flow

> "WebSockets start as HTTP. The client sends a special Upgrade header, the server agrees, and the connection is promoted to a persistent bidirectional channel. After that point, HTTP is no longer involved."

Short Polling

Client asks every N seconds. Simple to implement but wastes bandwidth even when nothing changed. Latency equals the polling interval.

HIGH OVERHEAD

Long Polling

Server holds the connection open until data changes. Lower latency than short polling, but server holds many open connections — resource-heavy with complex reconnection logic.

RESOURCE HEAVY

WebSockets

One-time HTTP handshake upgrades to a persistent bidirectional channel. Sub-millisecond latency. Either side can initiate — no request/response cycle required.

BEST FOR REAL-TIME

BLOCK 2 · 30 MIN

WebSockets vs HTTP

Protocol Comparison & When to Use Each

Property	HTTP (REST)	WebSocket
Connection	Opens and closes per request	Stays open
Direction	Client initiates only	Both sides can initiate
Overhead	Headers on every request	Headers on handshake only
State	Stateless	Stateful (server tracks connections)
Use case	CRUD, file transfer, caching	Live updates, chat, collaboration
Failure model	Server returns error code	Connection drops, client must reconnect

Use REST when the client asks a question. Use WebSockets when the server needs to speak first.

BLOCK 2 · WEBSOCKETS VS HTTP

When NOT to Use WebSockets

WebSockets are powerful, but they are not the right tool for every problem. Understanding the tradeoffs is more important than knowing the API.

Simple CRUD with no live audience

A personal note-taking app does not need sockets.

Large file transfers

HTTP with range headers handles this better.

Public APIs

REST is cacheable, well-documented, and stateless. Sockets are neither.

Serverless deployments

Persistent connections conflict with the function-per-request model of most serverless platforms.

Use REST when the client asks a question. Use WebSockets when the server needs to speak first.

BLOCK 3 · 45 MIN

Socket.IO Core Concepts

What Socket.IO Adds on Top of WebSockets

Raw WebSockets are a low-level protocol. Socket.IO is a library that builds on top of them and adds the features real applications need.

Socket.IO is NOT a thin wrapper around WebSockets. It has its own protocol on top. A Socket.IO server cannot communicate with a plain WebSocket client and vice versa. Always use the Socket.IO client library on the browser side.

Feature	Raw WebSocket	Socket.IO
Named events	No (raw binary frames)	Yes — emit('task:created', data)
Auto-reconnection	No	Yes — built-in with backoff
Rooms / channels	No	Yes — socket.join('room-name')
Fallback to polling	No	Yes (for restrictive networks)
Acknowledgements	No	Yes — callback-based confirmations
Broadcasting	Manual	Yes — io.emit, socket.broadcast.emit

BLOCK 3 · SOCKET.IO CORE CONCEPTS

The Event Model

Socket.IO is entirely event-driven. Everything is either emitting or listening to a named event.

```
// SERVER — emit to one client
socket.emit('task:created', newTask);
```

```
// SERVER — emit to ALL connected clients
io.emit('task:created', newTask);
```

```
// SERVER — emit to all EXCEPT the sender
socket.broadcast.emit('task:created', newTask);
```

```
// CLIENT — listen for an event
socket.on('task:created', (task) => {
  console.log('New task received:', task);
});
```

Event Naming Convention: resource:action

1. task:created
2. task:updated
3. task:deleted
4. user:joined
5. user:left

Avoid generic names like 'update' or 'data' — they become impossible to debug.

BLOCK 3 · SOCKET.IO CORE CONCEPTS

Connection Lifecycle

Every client connection fires a 'connection' event on the server. Each socket gets a unique ID. When the client disconnects, a 'disconnect' event fires with a reason.

```
// server.js
io.on('connection', (socket) => {
  // Fires once for each new client
  console.log('Client connected:', socket.id);

  // socket.id is a unique string, e.g. "xG3aBcD7..."

  socket.on('disconnect', (reason) => {
    // Fires when client closes tab, loses network, etc.
    console.log('Client disconnected:', socket.id, 'Reason:', reason);
  });
});
```

Reason	Meaning
transport close	Client tab closed or navigated away
ping timeout	Network went away — client didn't respond to heartbeat
server namespace disconnect	Server forcibly disconnected the client
transport error	Low-level network error

BLOCK 4 · 45 MIN

Integrating Socket.IO with the Task Board

The Architecture Decision

Before writing code, understand what changes and what stays the same. The REST API is untouched. Socket.IO is layered on top of the same HTTP server.

BEFORE (Day 3)

- server-express.js — HTTP server
- routes/taskRoutes.js — REST endpoints
- services/taskService.js — Business logic
- storage/fileStorage.js — Persistence

AFTER (Day 4)

- server-express.js — HTTP server + Socket.IO attached to same port
- routes/taskRoutes.js — REST endpoints (unchanged)
- services/taskService.js — Business logic (unchanged)
- storage/fileStorage.js — Persistence (unchanged)
- **socket/taskSocket.js** — NEW: socket event handlers
- **public/index.html** — NEW: browser client

The REST API is untouched. Socket.IO is layered on top of the same HTTP server.

BLOCK 4 · INTEGRATION

Attaching Socket.IO to the Express Server

Socket.IO wraps Node's http.Server, not Express directly. Always create the http.Server manually and pass it to both Express and Socket.IO.

```
// server-express.js (updated)
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const taskRoutes = require('./routes/taskRoutes');
const { logRequest } = require('./logger');

const app = express();
const httpServer = http.createServer(app); // wrap app in http.Server
const io = new Server(httpServer, {
  cors: { origin: '*' } // allow browser connections during development
});

// Middleware (unchanged from Day 3)
app.use(express.json());
app.use(express.static('public')); // serve the browser client
app.use((req, _res, next) => { logRequest(req); next(); });

// REST routes (unchanged from Day 3)
app.use('/tasks', taskRoutes);

// Socket.IO handlers
require('./socket/taskSocket')(io);

// Use httpServer.listen, not app.listen
const PORT = 3000;
httpServer.listen(PORT, () =>
  console.log(`Server ready on http://localhost:${PORT}`)
);
```

Common Mistake: Calling app.listen() instead of httpServer.listen(). Express creates its own internal http.Server when you call app.listen(), which is a different server object from the one Socket.IO is attached to. The result: Socket.IO connections never arrive.

BLOCK 4 · INTEGRATION

The Socket Handler Module

socket/taskSocket.js

All socket event handlers live in a dedicated module. This keeps server-express.js clean and mirrors the separation of concerns from the REST layer.

```
// socket/taskSocket.js
const taskService = require('../services/taskService');

module.exports = function registerTaskSocket(io) {

  io.on('connection', (socket) => {
    console.log(`Client connected: ${socket.id}`);

    // Client requests current task list on connect
    socket.on('tasks:request', async () => {
      try {
        const tasks = await taskService.getTasks();
        socket.emit('tasks:list', tasks); // send only to this client
      } catch (err) {
        socket.emit('error', { message: 'Failed to load tasks' });
      }
    });

    // Client creates a task
    socket.on('task:create', async ({ title }) => {
      try {
        if (!title?.trim()) {
          return socket.emit('error', { message: 'Title is required' });
        }
        const newTask = await taskService.addTask(title);
        io.emit('task:created', newTask); // broadcast to ALL clients
      } catch (err) {
        socket.emit('error', { message: 'Failed to create task' });
      }
    });

    // Client deletes a task
    socket.on('task:delete', async ({ id }) => {
      try {
        await taskService.deleteTask(id);
        io.emit('task:deleted', { id });
      } catch (err) {
        const msg = err.message === 'Task not found'
          ? 'Task not found'
          : 'Failed to delete task';
        socket.emit('error', { message: msg });
      }
    });

    // Client marks a task done/undone
    socket.on('task:toggle', async ({ id }) => {
      try {
        const tasks = await taskService.getTasks();
        const task = tasks.find(t => t.id === id);
        if (!task) return socket.emit('error', { message: 'Task not found' });
        task.done = !task.done;
        await taskService.saveTasks(tasks);
        io.emit('task:updated', task); // broadcast the updated task
      } catch (err) {
        socket.emit('error', { message: 'Failed to update task' });
      }
    });

    socket.on('disconnect', (reason) => {
      console.log(`Client disconnected: ${socket.id} — ${reason}`);
    });
  });
};
```

BLOCK 5 · 30 MIN

Rooms, Namespaces & Error Handling

Organising Connections

Rooms

Named channels that sockets can join or leave. Broadcasting to a room only reaches clients in that room — not all connected clients.

Use case: Multiple isolated groups (boards, channels, chat rooms)

Acknowledgements

Acknowledgements let the client confirm the server received and processed an event — like a lightweight response in HTTP.

Use case: Confirming task creation, validating inputs before broadcast

Namespaces

A separate Socket.IO server on the same HTTP server. Most apps don't need them — rooms are sufficient.

Use case: Isolating /admin vs /board logic, per-section middleware

For most applications, rooms are all you need. Namespaces are an advanced feature for complex multi-section apps.

BLOCK 5 · ROOMS, NAMESPACES & ERROR HANDLING

Rooms & Acknowledgements in Code

Rooms

```
// Client joins a room
socket.on('board:join', (boardId) => {
  socket.join(boardId);
  console.log(`${socket.id} joined room: ${boardId}`);
});

// Emit to everyone in a room (including sender)
io.to('board-123').emit('task:created', newTask);

// Emit to everyone in a room EXCEPT the sender
socket.to('board-123').emit('task:created', newTask);

// Leave a room
socket.leave(boardId);
```

Acknowledgements

```
// Client emits with a callback
socket.emit('task:create', { title: 'Buy milk' }, (response)
=> {
  if (response.error) {
    console.error('Server rejected:', response.error);
  } else {
    console.log('Task confirmed:', response.task);
  }
});

// Server calls the callback to acknowledge
socket.on('task:create', async ({ title }, ack) => {
  try {
    const newTask = await taskService.addTask(title);
    io.emit('task:created', newTask);
    ack({ task: newTask }); // success
  } catch (err) {
    ack({ error: err.message }); // error
  }
});
```

HANDS-ON LABS

Build the Collaborative Task Board

Four progressive labs that take you from a basic Socket.IO connection all the way to a fully collaborative, real-time task board with persistence.

Lab 1 — Install and Connect Socket.IO

Attach Socket.IO to the Day 3 Express server. Verify two browser tabs connect with distinct socket IDs.

Duration: ~15 min

Lab 2 — Real-Time Task Creation

Emit task:create from the browser, persist via the service layer, broadcast task:created to all clients.

Duration: ~20 min

Lab 3 — Collaborative UI (Delete + Toggle)

Add delete and toggle-done. All connected clients see every change in real time.

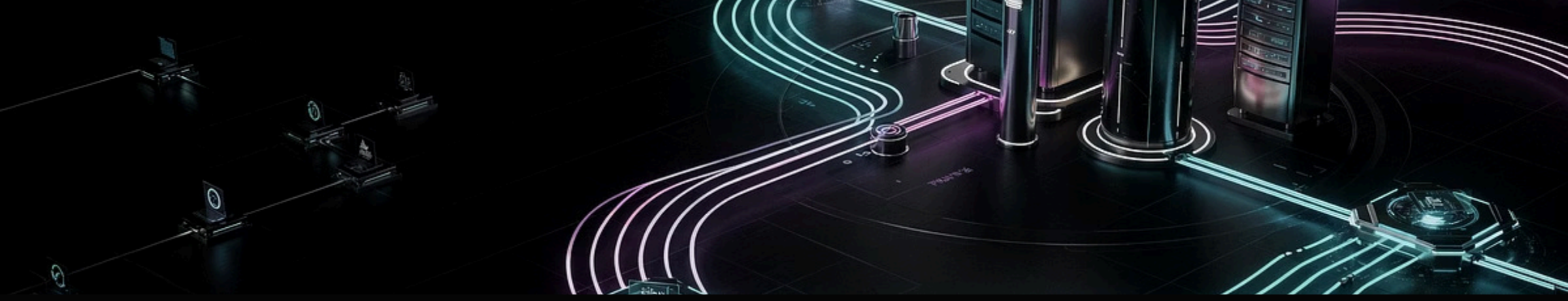
Duration: ~25 min

Lab 4 — Initial State on Connect

Ensure late-joining clients receive the current task list via a tasks:request / tasks:list pattern.

Duration: ~15 min

Optional Challenge: Real-Time Notifications — show who triggered each action using socket.id



LAB 1 · ~15 MIN

Install and Connect Socket.IO

Goal: Attach Socket.IO to the existing Day 3 Express server and verify a browser can connect.

01

Step 1: Install Socket.IO

Run: npm install socket.io

02

Step 2: Modify server-express.js

Wrap Express in http.createServer() and create a Server instance. Keep all existing middleware and routes exactly as they are.

03

Step 3: Add connection handler

Add a basic connection handler that logs each client's socket.id to the terminal.

04

Step 4: Create public/index.html

Add the Socket.IO client script tag and a console.log on connect.

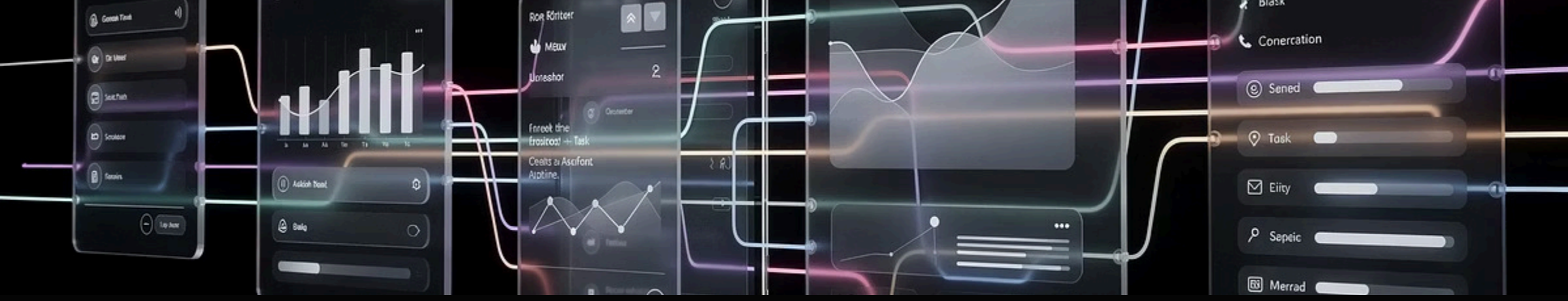
05

Step 5: Open two browser tabs

Open two tabs at http://localhost:3000. Verify the server logs two distinct IDs.

Expected terminal output:

```
Server ready on http://localhost:3000
Client connected: xG3aBcD7...
Client connected: mP9qRsT2...
```



LAB 2 · ~20 MIN

Real-Time Task Creation

Goal: Emit a task:create event from the browser, persist via the service layer, and broadcast task:created to all connected clients.

01

Step 1: Create socket/taskSocket.js

Require taskService. Register the task:create handler.

02

Step 2: Call taskService.addTask(title)

Inside the handler, use async/await to persist the new task through the service layer.

03

Step 3: Broadcast on success

Call io.emit('task:created', newTask) to broadcast to everyone.

04

Step 4: Handle errors

Call socket.emit('error', { message: ... }) to notify only the sender on failure.

05

Step 5: Build the browser UI

Add a text input, a button, and a socket.emit('task:create', ...) call on click.

06

Step 6: Listen and render

Listen for task:created and append the new task to the DOM.

Test: Open two tabs. Add a task in tab 1. Verify it appears in both tabs instantly — without a page refresh.

LABS 3 & 4

Collaborative UI + Initial State

Lab 3 · ~25 min — Delete + Toggle

Goal: Add delete and toggle-done functionality so all connected clients see every change.

- Add task:delete and task:toggle handlers in taskSocket.js
- For delete: remove via taskService.deleteTask(id), broadcast io.emit('task:deleted', { id })
- For toggle: flip task.done, save via taskService.saveTasks(tasks), broadcast io.emit('task:updated', task)
- Wire up delete button and title-click in the browser client
- Handle task:deleted (remove li from DOM) and task:updated (add/remove strikethrough)

Test: Open 2-3 tabs. Actions in one tab update all others in real time. Reload a tab — data persists.

Lab 4 · ~15 min — Initial State on Connect

Goal: Ensure a client that connects after tasks already exist receives the current list.

- Add a tasks:request handler that loads all tasks and emits tasks:list back to the requesting socket only (socket.emit, not io.emit)
- On the client, emit tasks:request immediately after the connect event fires
- Listen for tasks:list and render all received tasks

io.emit broadcasts only to clients connected at the time of emission. A client that connects after tasks were created would otherwise see an empty board. The request/response pattern over sockets solves this without changing the REST API.

INSTRUCTOR NOTES

Common Student Mistakes

Using app.listen() instead of httpServer.listen()

The most frequent error. When students call app.listen(), Express creates an internal server that Socket.IO never receives. Fix: require http, wrap app with http.createServer(app), call httpServer.listen().

Confusing io.emit, socket.emit, and socket.broadcast.emit

Draw this on the board:

- **io.emit('event')** → all clients (including sender)
- **socket.emit('event')** → only this client
- **socket.broadcast.emit('event')** → all clients EXCEPT this one
- **io.to('room').emit('event')** → all clients in that room

Forgetting that sockets and REST share the same data layer

Students sometimes add in-memory state inside taskSocket.js, causing sockets and REST to diverge. Reinforce that all mutations go through taskService.

The task:toggle race condition

Two clients clicking the same task simultaneously could both read done: false, both flip to done: true, and both write back — one write is lost. Acknowledge this is a real concurrency issue. Production systems use databases with atomic update operators.

TIPS

Pro Tips for Working with Socket.IO

Test with Multiple Browser Tabs

Always open two or more browser tabs when developing real-time features. Seeing both tabs update simultaneously is the fastest way to verify your socket logic is working correctly.

Use Chrome DevTools Network Tab

Filter by WS (WebSocket) in the Network tab. You can watch frames flowing in real time between client and server — invaluable for debugging event names and payloads.

Keep Sockets as Transport Only

Never put business logic or data storage inside your socket handlers. All mutations should go through the service layer — sockets are just the delivery mechanism.

socket.id is Not a User Identity

socket.id changes on every reconnect and is not tied to a real user. For authenticated apps, you'll need to pass a JWT through io.use() middleware. For now, socket.id is a useful stand-in.

DAY 4 OUTCOMES

What You've Built Today

Students leave with a working collaborative task board and a solid understanding of real-time architecture.

Working Collaborative Task Board

A real-time task board that updates across multiple browser tabs simultaneously.

WebSocket Protocol Understanding

How WebSockets differ from HTTP and when to use each.

Socket.IO Event Model

Hands-on experience with emit, on, broadcast, and rooms.

Hybrid Architecture

REST handles persistence. Sockets handle live updates. Both coexist cleanly.

Untouched Service Layer

The service and storage layers from Day 3 are unchanged — proving good architecture extends cleanly.

Foundation for Day 5

Ready for debugging, monitoring, and production practices.

> Next up: Day 5 — Debugging, Monitoring & Production Practices